CAT
TeamViewer Sync

Guía técnica y de usuario

Del correo en texto plano a TeamViewer, con reglas explícitas, modo simulación y sin sorpresas.

Versión 0.2.0

13 de octubre de 2026

CONTENIDO

Índice

1 · Resumen del proyecto qué problema resuelve y qué aporta

2 · Cómo funciona arquitectura y ciclo de vida de un lote

3 · Guía de usuario paso a paso, con capturas

4 · Guía técnica instalación, configuración, comandos, reglas, despliegue en una VM

5 · Seguridad modelo de amenazas, controles y auditoría

6 · Calidad y decisiones de diseño pruebas, CI, ADR y hoja de ruta

7 · Solución de problemas

Anexos glosario y referencia rápida

> **CÓMO LEER ESTA GUÍA** Si solo vas a *usar* CAT, lee los capítulos 1 y 3. Si vas a *instalarlo o mantenerlo*, lee el 2, el 4 y el 5. Todos los datos de los ejemplos (terminales, IDs, grupos) son ficticios.

CAPÍTULO 1

Resumen del proyecto

El problema

Las terminales que hay que dar de alta o modificar en TeamViewer llegan por **correo en texto plano**: una línea por terminal, con el formato que cada persona prefiera. Aplicarlas a mano es lento y propenso a errores: un grupo mal escrito, un número duplicado, o una terminal *reemplazada* (ID de TeamViewer nuevo, mismo número de terminal) dejan la libreta de direcciones inconsistente.

La solución

CAT automatiza ese trabajo manteniendo siempre a una persona en el circuito:

1. **Interpreta** el correo pegado en una interfaz web y extrae sucursal, número de terminal, ID de TeamViewer, versión, tipo y grupo.
2. **Revisa**: la persona corrige lo que haga falta en una tabla editable; el sistema valida y marca en rojo lo que está mal.
3. **Encola**: el lote confirmado se guarda como un archivo JSON en un repositorio privado de GitHub (la *cola*).
4. **Aplica**: un proceso (*worker*) lee la cola y, para cada terminal, un **motor de reglas** decide qué hacer en TeamViewer: crear, renombrar, mover, reemplazar o pedir revisión manual.

Qué aporta

9
reglas de negocio explícitas (R100 a R401), cada una con su prueba

100
pruebas automatizadas, sin acceso a red ni a cuentas reales

0
vulnerabilidades conocidas (pip-audit, npm audit) y 0 hallazgos de bandit

Característica	Beneficio
Modo simulación	`--dry-run` y `--file` muestran qué haría el programa sin tocar TeamViewer ni la cola.
Falla cerrado	Si TeamViewer no responde, no se aplica nada y el lote queda pendiente. Nunca se evalúa contra una lista vacía (eso crearía duplicados).
Nunca pisa un equipo vivo	Si el número de terminal ya está *online* en otra máquina, se detiene y pide revisión manual (regla R300).
Trazabilidad	Cada terminal queda con su resultado (regla, acción, estado, mensaje) en el JSON archivado.
Errores visibles	Las filas inválidas se marcan en rojo, con su número y un mensaje en español.
Seguro por defecto	La web escucha solo en el propio equipo; para exponerla exige contraseña.

CAPÍTULO 2

Cómo funciona

Arquitectura

Son dos procesos que se comunican **solo a través de la cola**: la interfaz web y el worker. Eso permite que la web corra en cualquier equipo mientras un único worker controla TeamViewer.

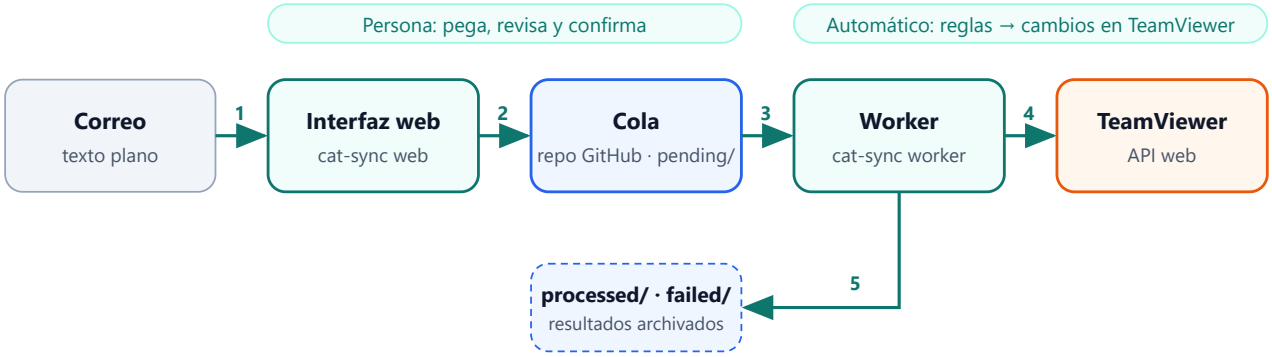


Figura 1. Flujo de un lote: ① se pega el correo, ② se valida y se guarda como JSON en la cola, ③ el worker lo lee, ④ aplica los cambios en TeamViewer, ⑤ archiva el resultado.

Componentes

Módulo	Responsabilidad
`engine.py`	**Motor de reglas**: función pura *(terminal pedida, equipos actuales) → decisión*. Sin entrada/salida, por eso es fácil de probar.
`executor.py`	Convierte una decisión en llamadas a la API; en simulación no envía nada.
`worker.py`	Bucle de la cola, resultados por terminal, y a dónde se archiva cada lote.
`teamviewer.py`	Cliente de la API de TeamViewer (grupos y equipos), con reintentos.
`github_queue.py`	La cola sobre la API de contenidos de GitHub.
`parser.py` · `validators.py`	Del texto del correo a filas; validación antes de encolar.
`web/app.py`	Backend FastAPI; sirve también la interfaz compilada.
`frontend/`	Interfaz React + Vite (tabla editable, errores en rojo).

Ciclo de vida de un lote

1. La web escribe `pending/2026-10-13_10-30-00_ab12.json` en el repositorio de la cola; el worker consulta esa carpeta cada `POLLING_INTERVAL` segundos (60 por defecto), del lote más antiguo al más nuevo.
2. Lee el estado actual de TeamViewer. **Si falla, el ciclo se detiene y el lote queda pendiente.**
3. Para cada terminal: valida el registro, el motor decide, el ejecutor aplica y el estado en memoria se actualiza (la siguiente terminal ve el efecto de la anterior).
4. Archiva el lote con su resultado: a `processed/` si todo salió bien (la revisión manual es un resultado normal) o a `failed/` si hubo errores.

CAPÍTULO 3

Guía de usuario

3.1 Antes de empezar

Necesitas que alguien (normalmente quien instala CAT) haya dejado configurado el archivo `.env` con los accesos, y la lista de grupos válidos en `groups.json`. Si ya puedes abrir `start-cat.bat` sin errores, estás listo.

3.2 Abrir CAT

1. Haz doble clic en `start-cat.bat`. Se abre **una sola ventana** negra: no la cierres mientras trabajes.
2. Se abre solo el navegador en `http://127.0.0.1:8000`. La pestaña se llama **CAT**.
3. La primera vez puede tardar unos minutos porque prepara el entorno; las siguientes veces es inmediato.

> **AVISO** Si al abrir ves el mensaje *«Modo SIMULACIÓN activo»*, el worker solo muestra qué haría y **no modifica TeamViewer**. Es la configuración segura para probar. Quien administra CAT la desactiva cuando todo está validado.

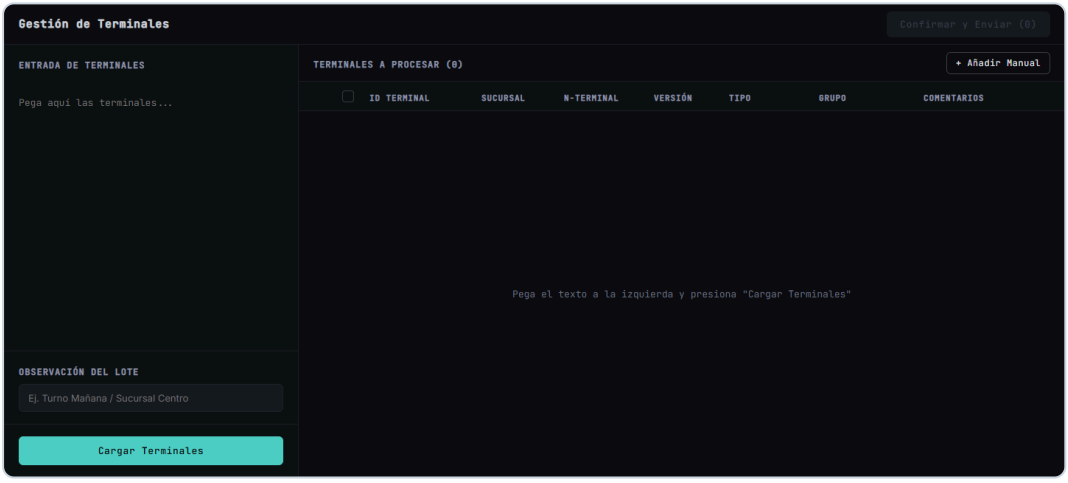


Figura 2. Pantalla inicial. A la izquierda se pega el correo; a la derecha aparecerá la tabla de terminales.

3.3 Cargar las terminales

1. Copia el texto del correo y pégalo en el panel izquierdo **«Entrada de terminales»**. Puedes pegar también las líneas de saludo: CAT las ignora.
2. Pulsa **«Cargar Terminales»**. En un instante aparece la tabla con una fila por terminal y el aviso verde *«N terminal(es) detectada(s) correctamente»*.

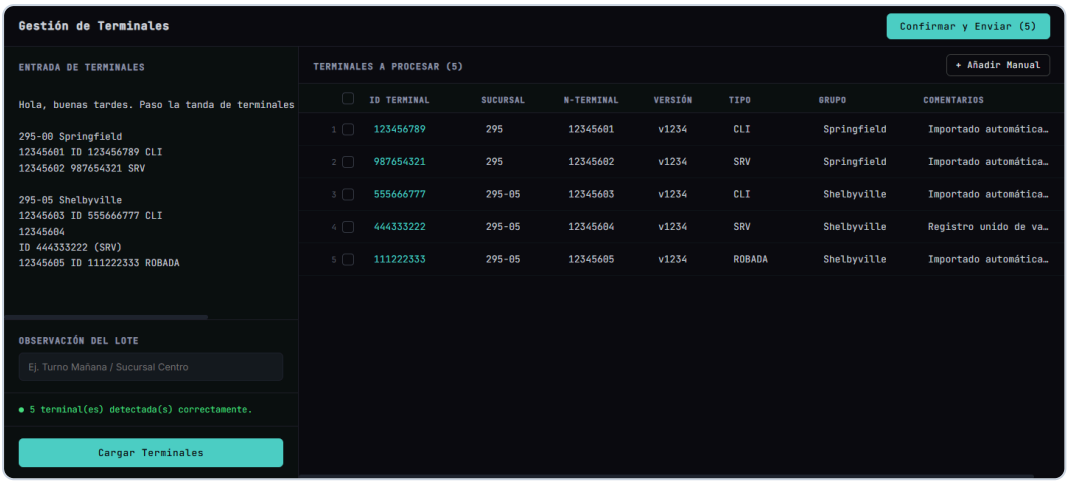


Figura 3. El correo ya interpretado. Observa que `295-00` quedó como `295` y que `295-05` conserva su sufijo.

3.4 Revisar y corregir

La tabla es editable: **haz clic en cualquier celda** para corregirla (Enter confirma, Esc cancela). La columna *Grupo* se elige de una lista. Cada fila tiene su **número** a la izquierda; los mensajes de error usan esos mismos números.

Columna	Qué es
ID Terminal	El ID de TeamViewer, de 8 a 11 dígitos.
Sucursal	Código de sucursal (por ejemplo `295` o `295-05`). El sufijo `-00` se elimina porque solo indica «sin detalle de localidad».
N-Terminal	Número de la terminal.
Versión	Por ejemplo `v1234`. Si el correo no la trae, queda vacía para que la completes.
Tipo	`CLI` (cliente), `SRV` (servidor), `ROBADA` o «Sin tipo».
Grupo	Debe ser uno de la lista. Si CAT no reconoce el grupo del correo, lo deja **vacío** para que lo elijas tú.

- **+ Añadir Manual** agrega una fila vacía. La **×** de cada fila (al pasar el mouse) la elimina.
- Marca casillas para enviar **solo las filas seleccionadas**; si no marcas ninguna, se envían todas.

3.5 Cuando hay errores

Al pulsar **«Confirmar y Enviar»**, CAT valida todo antes de guardar nada. Si encuentra problemas, **marca en rojo las filas afectadas**, muestra cuántas son junto al título de la tabla, escribe el motivo abajo a la izquierda y se desplaza hasta la primera.



Figura 4. El ID `123456789` está repetido en las filas 1 y 6: ambas se marcan en rojo y el mensaje indica «Fila 6: … se repite con la fila 1».

Mensaje	Qué hacer
El ID de TeamViewer … se repite con la fila N	Una de las dos filas está mal; corrige el ID o elimina la duplicada.
La terminal N aparece con IDs distintos	El mismo número con dos IDs en el mismo lote: decide cuál es el correcto.
Grupo desconocido / elige un grupo válido	Elige el grupo en la lista de la celda.
«X» es obligatorio	Completa la celda vacía.
El ID debe tener entre 8 y 11 dígitos	Revisa que el ID esté completo (una fila «FALTA ID» no tenía ID en el correo).

Al editar una fila, su marca roja desaparece. Corrige todas y vuelve a enviar.

3.6 Enviar el lote

Con todo correcto, pulsa **«Confirmar y Enviar»**. El lote se guarda en la cola y la tabla se vacía con el aviso *«N terminal(es) enviada(s) exitosamente»*. Opcionalmente puedes escribir una nota en **«Observación del lote»** (por ejemplo «Turno mañana»).

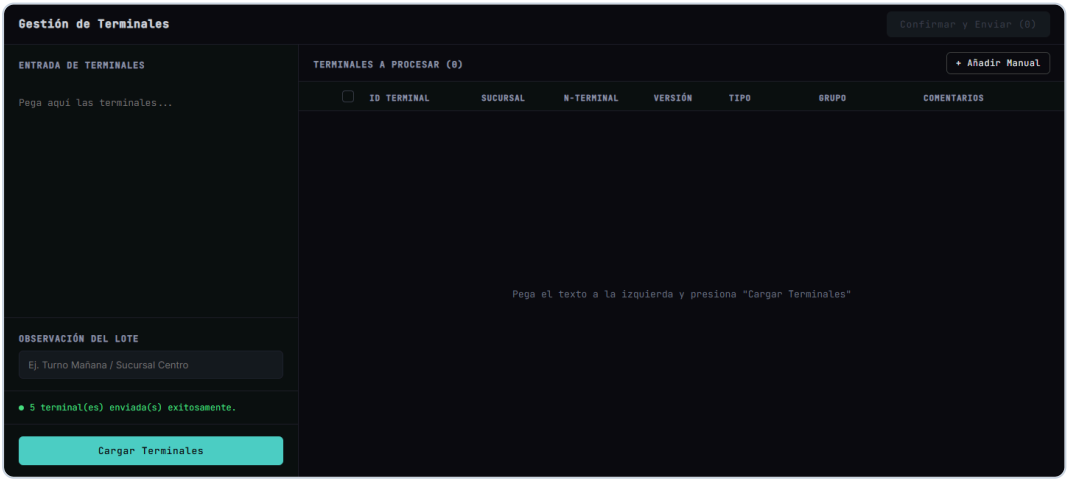


Figura 5. Lote enviado. (Captura de documentación: el envío se simuló para no escribir en una cola real.)

3.7 Qué pasa después

El *worker* (la ventana de CAT sigue abierta) toma el lote, aplica los cambios y archiva el resultado. Tú no tienes que hacer nada más, salvo revisar dos casos:

Resultado	Significado y acción
OK / SIN CAMBIOS	La terminal se creó o actualizó, o ya estaba correcta.
REVISIÓN MANUAL	El número de terminal ya está **online** en otra máquina. CAT no toca nada: alguien debe decidir cuál es la correcta.
ERROR	Falló alguna llamada o el registro era inválido. El lote pasa a la carpeta `failed/` del repositorio, con el detalle por terminal.

3.8 Formato de correo que entiende CAT

```
Hola, buenas tardes. Paso la tanda de terminales de hoy, versión v1234.

295-00 Springfield                    ← sucursal + grupo: fija el contexto de las líneas siguientes
12345601 ID 123456789 CLI             ← terminal + ID de TeamViewer + tipo
12345602 987654321 SRV                ← el ID puede ir sin la palabra «ID»

295-05 Shelbyville
12345604                              ← el ID puede venir en la línea siguiente
ID 444333222 (SRV)
```

- Una línea con **sucursal y grupo** (sin terminal) fija el contexto para las terminales que siguen.
- El número de terminal tiene el formato configurado (por defecto, 8 dígitos que empiezan con `12`); el ID de TeamViewer tiene de 8 a 11 dígitos.
- La versión (`v1234`) puede ir en cada línea o una sola vez en todo el correo.
- Si el grupo coincide con varios nombres, gana el **más largo** (por ejemplo «San Martín Norte» sobre «San Martín»).

3.9 Cerrar CAT

Cuando termines de cargar lotes y no haya nada pendiente, cierra la ventana de `start-cat.bat` (o pulsa **Ctrl + C**). Los lotes que queden en la cola se procesarán la próxima vez que lo abras. Si lo cierras en mitad de un lote, no pasa nada grave: la siguiente ejecución retoma el trabajo sin duplicar.

CAPÍTULO 4

Guía técnica

4.1 Requisitos

Elemento	Detalle
Sistema	Windows (con `start-cat.bat`), Linux o macOS (con `scripts/start.sh`)
Python	3.10 o superior
Node.js	Solo para compilar la interfaz una vez (`npm ci && npm run build`)
Cuenta de TeamViewer	Token de script con permiso para gestionar grupos y dispositivos
Repositorio de GitHub	Privado, con un token *fine-grained* de **Contents: read & write** solo sobre ese repositorio

4.2 Instalación

```
# 1. Obtener el código y crear el entorno
git clone <repositorio> && cd cat-teamviewer-sync
python -m venv .venv
.venv\Scripts\activate                # Linux/macOS: source .venv/bin/activate
pip install -e ".[web]"

# 2. Compilar la interfaz (una vez)
cd frontend && npm ci && npm run build && cd ..

# 3. Configurar
copy .env.example .env                # completa los tokens
copy groups.example.json groups.json  # tus nombres de grupo válidos

# 4. Verificar (no cambia nada)
cat-sync check
```

En Windows, `start-cat.bat` hace los pasos 1 y 2 por ti la primera vez.

4.3 Configuración

Todo se define con variables de entorno; un archivo `.env` en la carpeta del proyecto se lee automáticamente. El archivo `.env` está en `.gitignore` y nunca debe subirse.

Variable	Por defecto	Significado
`GITHUB_TOKEN`, `GITHUB_REPO`	—	Token y repositorio (`usuario/nombre`) de la cola
`GITHUB_BRANCH`	rama por defecto	Rama de la cola
`QUEUE_FOLDER_PENDING` / `_PROCESSED` / `_FAILED`	`pending` / `processed` / `failed`	Carpetas dentro del repositorio de la cola
`TV_API_TOKEN`	—	Token de script de TeamViewer
`TV_DEFAULT_PASSWORD`	vacío	Contraseña de acceso desatendido de los equipos nuevos (opcional)

Variable	Por defecto	Significado
`POLLING_INTERVAL`	`60`	Segundos entre consultas a la cola (mínimo 5)
`CAT_DRY_RUN`	`false`	`true`: el worker solo simula una pasada y no modifica nada
`LOG_FILE_PATH`	`logs/cat.log`	Archivo de registro (rota a 10 MB, 5 copias)
`CAT_TERMINAL_REGEX`	`12\d{6}`	Cómo luce un número de terminal en tus correos y en los alias
`CAT_GROUPS_FILE`	`groups.json`	Lista JSON de grupos permitidos
`CAT_WEB_HOST` · `CAT_WEB_PORT`	`127.0.0.1` · `8000`	Dónde escucha la web
`CAT_WEB_USER` · `CAT_WEB_PASSWORD`	`cat` · vacío	Credenciales HTTP Basic; **obligatoria** si la web escucha fuera de localhost
`CAT_ALLOWED_HOSTS`	`localhost,127.0.0.1`	Valores de `Host` aceptados (añade el nombre de tu VM si la expones)
`CAT_CORS_ORIGINS`	localhost	Orígenes permitidos por CORS

4.4 Línea de comandos

Comando	Para qué
`cat-sync web [--host H] [--port P]`	Inicia la interfaz web
`cat-sync worker`	Procesa la cola de forma continua
`cat-sync worker --once`	Una sola pasada y termina
`cat-sync worker --dry-run`	Simula los lotes en cola sin aplicar nada (no los consume)
`cat-sync worker --file LOTE.json`	Simula un archivo local contra el estado real de TeamViewer. Solo lectura; no usa la cola
`cat-sync check [--groups]`	Verifica credenciales y conectividad
`cat-sync backup [--out-dir D]`	Exporta todos los grupos y equipos de TeamViewer a un JSON

Salida de ejemplo de la simulación de un archivo local (datos ficticios):

```
$ cat-sync worker --file examples\batch_sample.json
2026-10-13 10:30:11 [INFO] Lote batch_sample.json: 2 registros, 240 equipos en TeamViewer
2026-10-13 10:30:11 [INFO]   [SIMULADO] R401 terminal 12345601: Alta nueva: se creará
    el grupo 'Springfield' y la terminal '101 - 12345601 - v1234 - CLI - Springfield'.
2026-10-13 10:30:11 [INFO]   [SIMULADO] R400 terminal 12345602: Alta nueva: se creará
    '101 - 12345602 - v1234 - SRV - Springfield' en el grupo 'Springfield'.

SIMULACIÓN de batch_sample.json: no se modificó nada en TeamViewer ni en la cola.
  crearía 2, actualizaría 0, sin cambios 0, revisión manual 0, errores 0
```

4.5 Reglas de negocio

El motor (`engine.py`) mira dos cosas: el equipo con el *mismo ID* y los *otros* equipos con el *mismo número de terminal pero distinto ID*. La **R300 se evalúa primero**: si el número está vivo en otra máquina, se detiene todo.

Regla	Acción	Condición	Efecto
R100	`DO_NOTHING`	El equipo existe y grupo y alias coinciden	Nada
R201	`MOVE_GROUP`	Existe, el grupo difiere	Mueve (creando el grupo si falta)
R202	`RENAME_TERMINAL`	Existe, el alias difiere	Renombra
R203	`MOVE_AND_RENAME`	Difieren ambos	Mueve y renombra
R300	`MANUAL_REVIEW`	El mismo número está **online** en otro equipo	No toca nada; lo reporta
R301	`REPLACE_SECURE`	ID nuevo; el número solo existe en equipos **offline**	Crea el nuevo y luego borra los viejos
R302	`DELETE_OFFLINE_DUPLICATES`	Equipo ya correcto, pero hay duplicados offline de su número	Borra los duplicados
R400	`CREATE_TERMINAL`	No existe y el grupo sí	Crea
R401	`CREATE_GROUP_AND_TERMINAL`	No existe y el grupo tampoco	Crea el grupo y luego la terminal

POR QUÉ EN ESE ORDEN Si se aplicara el ticket cuando el número está online en otra máquina, se le quitaría el control a un equipo en uso. Los borrados (R301, R302) ocurren *después* de que la acción principal tuvo éxito.

4.6 Convención de alias

```
<sucursal> - <número> - <versión> [- <tipo>] - <grupo>
101 - 12345601 - v1234 - CLI - Springfield      ← con tipo
101 - 12345603 - v1234 - Springfield            ← sin tipo («Sin tipo» no aparece)
```

4.7 Formato de la cola

Lote pendiente (lista JSON escrita por la web). Las claves se mantienen en español por compatibilidad:

```
[
  {
    "ID Teamviewer": "123456789",
    "Sucursal": "295",
    "N-Terminal": "12345601",
    "Versión": "v1234",
    "Tipo": "CLI",
    "Grupo": "Springfield",
    "Comentarios / Observaciones": "Importado automáticamente del correo",
    "Observacion_General": "Turno mañana"
  }
]
```

Lote archivado (`processed/` o `failed/`), escrito por el worker:

```
{
  "result": {
    "correlation_id": "CAT-20261013-A1B2C3",
    "processed_at": "2026-10-13T14:03:11+00:00",
    "dry_run": false,
    "metrics": { "created": 1, "updated": 0, "unchanged": 0, "manual_review": 0, "errors": 0 },
    "outcomes": [
      { "terminal": "12345601", "tv_id": "123456789", "rule_id": "R400",
        "action": "CREATE_TERMINAL", "status": "ok",
        "message": "Alta nueva: se creará '295 - 12345601 - v1234 - CLI - Springfield' en el grupo
'Springfield'." }
    ]
  },
  "terminals": [ "...los registros originales..." ]
}
```


`status` toma uno de estos valores: `ok`, `planned` (simulación), `unchanged`, `manual_review`, `error`. Para reintentar un lote en `failed/`, corrígelo y devuélvelo a `pending/`.

4.8 Estructura del proyecto

```
cat-teamviewer-sync/
├─ start-cat.bat            arranque con doble clic (Windows)
├─ src/cat_sync/
│   ├─ engine.py            reglas (puro)        ├─ models.py   Ticket · TvDevice · Decision
│   ├─ executor.py          decisión → API       ├─ naming.py   alias, sucursal, número
│   ├─ worker.py            cola, resultados     ├─ parser.py   correo → filas
│   ├─ teamviewer.py        cliente TeamViewer   ├─ validators.py
│   ├─ github_queue.py      cola en GitHub       ├─ config.py   variables de entorno
│   ├─ web/app.py           FastAPI              └─ cli.py      comando cat-sync
├─ frontend/                React + Vite
├─ tests/                   pytest, sin red
├─ docs/                    arquitectura, reglas, ADR, esta guía
└─ examples/                correo y lote ficticios
```

4.9 Despliegue en una VM

El patrón recomendado es dejar la carpeta completa en una VM y abrir `start-cat.bat` **solo cuando se necesita**. Mientras está cerrado no escucha nada, y los lotes que se hayan encolado esperan en GitHub.

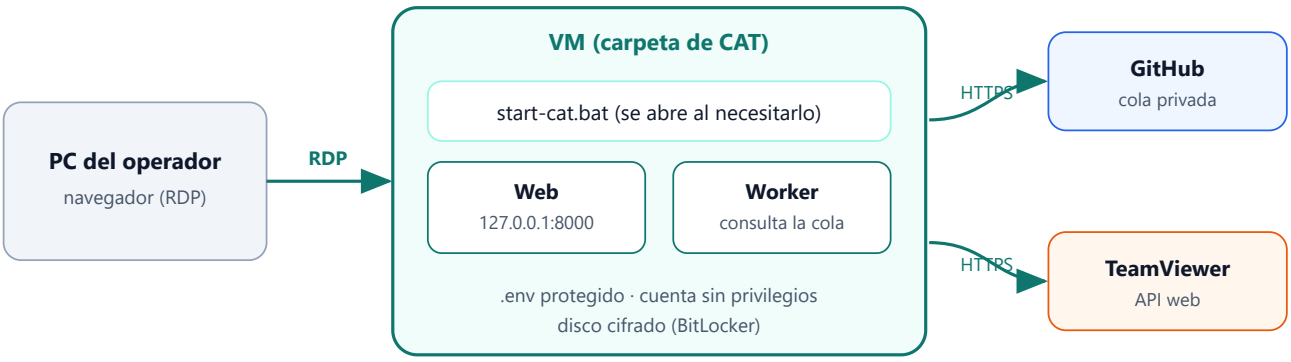


Figura 6. Despliegue recomendado: la persona entra a la VM por RDP y usa la web en `127.0.0.1`. Nada se expone a la red; las únicas conexiones salientes son HTTPS hacia GitHub y TeamViewer.

1. Copia la carpeta a la VM (sin `.venv` ni `node_modules`: no se pueden mover entre equipos) y ejecuta `start-cat.bat` una vez para preparar el entorno.
2. Crea `.env` y `groups.json`. Restringe sus permisos NTFS a una cuenta de servicio.
3. Valida con `cat-sync check` y con `cat-sync worker --file lote-real.json`.
4. Mantén `CAT_DRY_RUN=true` hasta confirmar un lote real; luego cámbialo a `false`.
5. Úsalo: abre `start-cat.bat`, carga los lotes, espera a que el worker los procese y ciérralo.

> **¿Y SI LO CIERRO CON TRABAJO A MEDIAS?** No se pierde nada: un lote no se archiva hasta terminarse y el motor es idempotente. Al abrirlo de nuevo, lo ya aplicado se detecta como R100 (sin cambios) y se limpia cualquier duplicado pendiente (R302).

4.10 Operación

Tarea	Cómo
Ver qué pasó	`logs/cat.log` y la carpeta `processed/` / `failed/` del repositorio de la cola
Reintentar un lote	Corrige el JSON de `failed/` y muévelo a `pending/`
Respaldo previo a cambios grandes	`cat-sync backup` (queda en `backups/`, ignorado por git; contiene datos reales de la cuenta)
Marcha atrás	Detén CAT y vuelve a `CAT_DRY_RUN=true`. Esto no deshace cambios ya aplicados: usa el respaldo para consultar el estado anterior
Añadir un grupo nuevo	Agrégalo a `groups.json`; CAT lo creará en TeamViewer en el primer alta (R401)

CAPÍTULO 5

Seguridad

5.1 Qué se protege

CAT puede **crear, renombrar, mover y borrar equipos** de una cuenta de TeamViewer. Quien controle sus entradas controla esos equipos, así que hay tres activos críticos: el **token de TeamViewer**, el **token de GitHub** de la cola (escribir en la cola equivale a mandar sobre TeamViewer) y la **interfaz web**, que es la puerta de entrada.

5.2 Controles implementados

Área	Control
Exposición de red	La web escucha en `127.0.0.1`. Escuchar en otra dirección **se niega a arrancar** si no hay `CAT_WEB_PASSWORD` (falla cerrado).
Autenticación	HTTP Basic opcional, con comparación en tiempo constante.
DNS rebinding	Lista blanca de cabeceras `Host` (`CAT_ALLOWED_HOSTS`).
Navegador	`Content-Security-Policy`, `X-Frame-Options: DENY`, `X-Content-Type-Options`, `Referrer-Policy`, `Cache-Control: no-store` en la API y CORS limitado a localhost.
Validación de entrada	Límites de tamaño en cada campo, IDs de 8 a 11 dígitos, sin caracteres de control (evita inyección en el log), lista blanca de grupos, detección de duplicados.
Cola no confiable	El worker **revalida cada registro** que lee: un archivo colocado a mano en el repositorio no puede introducir IDs o textos raros en TeamViewer.
Falla cerrado	Si no se puede leer TeamViewer, no se aplica nada; un equipo vivo nunca se pisa (R300).
Secretos	Solo en `.env` (ignorado por git); excluidos de `repr()`; nunca se registran. Los tokens viajan solo en cabeceras `Authorization`.
Automatización segura	Simulación sin efectos; los `POST` nunca se reintentan (no hay altas duplicadas).

5.3 Resultados de la auditoría

Herramienta	Qué revisa	Resultado
`pip-audit`	Vulnerabilidades conocidas en dependencias de Python	**Sin hallazgos**
`npm audit`	Vulnerabilidades conocidas en dependencias del frontend	**0 vulnerabilidades** (se corrigieron 2 de nivel alto y se eliminó Tailwind, que no se usaba)
`bandit`	Análisis estático de seguridad del código Python	**Sin hallazgos** (se corrigió un falso positivo B107)
Navegador real	Que las cabeceras (CSP) no rompan la interfaz	**0 violaciones de CSP**

5.4 Recomendaciones para la VM

- **Usa la web desde la propia VM** (RDP o consola) en `127.0.0.1`: no expone nada a la red. Es la opción más segura.
- Si otros equipos deben usarla, define `CAT_WEB_HOST`, `CAT_WEB_PASSWORD` y `CAT_ALLOWED_HOSTS` y colócala **dentro de una VPN o detrás de un proxy HTTPS**: HTTP Basic no va cifrado por HTTP plano.
- Ejecútalo con una **cuenta de Windows dedicada y sin privilegios de administrador**; limita el acceso a la carpeta y a `.env`; activa BitLocker.
- **Token de GitHub** *fine-grained*, con caducidad, limitado al repositorio de la cola; repositorio **privado**.
- **Rota los tokens** si alguna vez estuvieron en texto plano en un lugar compartido.
- Ejecuta `pip-audit` y `npm audit` periódicamente y mantén Windows y Python actualizados.

5.5 Limitaciones conocidas

> **IMPORTANTE**
>
> - La web usa **UNA SOLA CREDENCIAL COMPARTIDA** y no registra qué persona encoló cada lote (el historial de GitHub muestra que se encoló, no quién).
> - Quien tenga **ESCRITURA EN EL REPOSITORIO DE LA COLA** puede encolar terminales de aspecto válido: el worker rechaza registros mal formados, no registros no autorizados. Protege ese repositorio.
> - Sin HTTPS, la contraseña HTTP Basic viaja sin cifrar.
> - No hay límite de peticiones (rate limiting).
> - La página carga tipografías de Google Fonts; en una VM sin internet usa las del sistema.

CAPÍTULO 6

Calidad y decisiones de diseño

6.1 Pruebas

La suite tiene **100 pruebas automatizadas** (pytest) que no usan red ni cuentas reales: TeamViewer y la cola se sustituyen por equivalentes en memoria. Además se ejecutan `ruff` (estilo), `eslint` y la compilación del frontend. Un flujo de GitHub Actions repite todo en cada cambio, con Python 3.10, 3.12 y 3.13.

Archivo	Pruebas	Qué cubre
`test_engine.py`	17	Una prueba por regla (R100 a R401), prioridad de R300 y efecto sobre el estado dentro de un lote
`test_worker.py`	16	Simulación, revisión manual, reemplazo seguro, lotes fallidos, caída de TeamViewer, registros plantados en la cola
`test_clients.py`	13	Clientes de TeamViewer y GitHub con respuestas simuladas, reintentos y archivado
`test_web.py`	15	Endpoints, errores 400/502/503, filas con error, CORS, cabeceras, `Host`, contraseña, caché
`test_parser.py` · `test_validators.py` · `test_naming.py`	29	Correos de ejemplo, sufijo `-00`, validaciones y construcción de alias
`test_cli.py`	10	Simulación de archivo, negativa a exponerse sin contraseña, secretos fuera del `repr`

6.2 Decisiones de diseño

Decisión	Por qué
Un repositorio privado de GitHub como cola (ADR 0001)	Sin servidor ni base de datos que mantener; historial de auditoría gratuito; los lotes fallidos son archivos legibles y reparables. Costo aceptado: latencia de consulta y un solo worker.
Motor de reglas puro	Sin entrada/salida, cada regla es una función verificable por pruebas; añadir una regla no toca la red.
Los adaptadores lanzan excepciones	La primera versión devolvía listas vacías ante un fallo, lo que provocaba altas duplicadas. Ahora el error se propaga y el lote espera.
Persona en el circuito	El correo es ambiguo; la tabla editable y la validación evitan errores antes de tocar TeamViewer.
Código en inglés, mensajes en español	El código es reutilizable y legible internacionalmente; lo que ve el operador está en su idioma. Los valores técnicos (`status`) no se traducen.

6.3 Hoja de ruta

- Pantalla de resultados en la web (qué se aplicó y qué requiere revisión manual).
- Aviso cuando una terminal queda en revisión manual.
- Usuarios individuales con registro de quién encola cada lote.
- Servicio de Windows / unidad systemd para el worker.
- Tipografías incluidas en el paquete, sin depender de Google Fonts.

CAPÍTULO 7

Solución de problemas

Síntoma	Causa probable	Qué hacer
El `.bat` se cierra solo	Falta `.env`, no hay Python, o el entorno se copió desde otro equipo	Ábrelo desde una consola para ver el mensaje; crea `.env`; borra `.venv` y vuelve a abrirlo para recrearlo
Falta configuración: GITHUB_TOKEN…	No hay `.env` en la carpeta o está incompleto	Copia `.env.example` a `.env` y rellena los valores
No se pudo conectar con el servidor	La ventana de `start-cat.bat` está cerrada	Ábrela de nuevo y recarga la pestaña
La interfaz muestra algo viejo o `[object Object]`	El navegador guardó una versión anterior	**Ctrl + F5** una vez; luego el servidor evita que vuelva a pasar
Una fila en rojo con *grupo desconocido*	El grupo no está en `groups.json`	Elige uno de la lista, o añádelo al archivo y reinicia
Para escuchar fuera de localhost define CAT_WEB_PASSWORD	Se usó `--host` distinto de localhost sin contraseña	Define la contraseña, o usa la web desde la propia VM
Invalid host header (400)	Se accede por un nombre no incluido en `CAT_ALLOWED_HOSTS`	Añade el nombre de la VM a esa variable
Nada se aplica en TeamViewer	`CAT_DRY_RUN=true`	Es el modo simulación; ponlo en `false` cuando esté validado
Terminal en `[REVISIÓN MANUAL]`	R300: el número está online en otra máquina	Decide cuál es la correcta; corrige el origen y reenvía
Un lote aparece en `failed/`	Al menos un registro tuvo error	Lee el `outcomes` del JSON, corrige y devuélvelo a `pending/`
El puerto 8000 está ocupado	Otra instancia de CAT abierta	Ciérrala o usa `--port`

ANEXOS

Glosario y referencia rápida

A · Glosario

Término	Significado
Terminal	Equipo que se registra en TeamViewer, identificado por su *número de terminal*.
ID de TeamViewer	Identificador numérico (8 a 11 dígitos) del equipo en TeamViewer. Puede cambiar si se reinstala; el número de terminal no.
Alias	El nombre con el que se ve el equipo en TeamViewer (ver 4.6).
Lote	Conjunto de terminales confirmadas juntas; se guarda como un archivo JSON.
Cola	La carpeta `pending/` del repositorio privado de GitHub.
Worker	Proceso que lee la cola y aplica los cambios en TeamViewer.
Simulación (dry run)	Ejecución que calcula y muestra los cambios sin aplicarlos.
Online / offline	Estado de conexión del equipo en TeamViewer en el momento de procesar el lote.
Idempotente	Repetir la operación no cambia el resultado: volver a procesar un lote no duplica nada.

B · Referencia rápida

```
# Usar
start-cat.bat                                arrancar todo (Windows)

# Verificar y probar sin riesgo
cat-sync check                               credenciales y conectividad
cat-sync worker --file lote.json             simular un archivo local
cat-sync worker --dry-run                    simular los lotes en cola

# Operar
cat-sync worker                              procesar la cola de forma continua
cat-sync worker --once                       una sola pasada
cat-sync backup                              respaldar grupos y equipos de TeamViewer
cat-sync web --port 8000                     solo la interfaz web
```